

Networking Notes

concepts + real-world troubleshooting, for exams / interviews / DevOps

1. What is Networking?

Communication between devices to exchange data & access resources.

Big data is broken into **packets**, sent across the network, and reassembled at destination.

Sender → Data → Packets → Network → Receiver → Reassembled Data

2. Types of Networks

Network	Meaning	Scope
PAN	Personal Area Network	Personal devices
LAN	Local Area Network	Home / office
MAN	Metropolitan Area Network	City
WAN	Wide Area Network	Large geography
CAN	Campus Area Network	University / company

3. Network Devices

NIC — Network Interface Card

Lets a device connect to a network. Tied to a **MAC address**.

Hub (Layer 1)

- Receives data
- Broadcasts to ALL ports
- Mostly obsolete now

A → HUB → B
→ C
→ D

Switch ☆ (Layer 2)

- Connects devices within a LAN
- Uses MAC addresses
- Forwards frames to intended device

Router ☆ (Layer 3)

- Connects different networks
- Uses IP addresses to forward packets

Switch → within LAN, using MAC | Router → across networks, using IP

4. MAC vs IP Address ★★★★★

MAC Address

- Tied to the network interface
- Local / Layer-2 communication
- Used by switches
- Hexadecimal, e.g. 00:1A:2B:3C:4D:5E

IP Address

- Logical network address
- Used for routing
- Used by routers
- IPv4 = 32 bits, IPv6 = 128 bits

interview Q: why need both?

IP → tells where a device is logically. MAC → identifies the interface for local delivery.

5. OSI Model ★★★★★

Mnemonic → All People Seem To Need Data Processing

#	Layer	Deals with
7	Application	HTTP, DNS, SSH, SMTP, gRPC
6	Presentation	TLS, encoding, serialization, compression
5	Session	Establish / maintain / terminate sessions
4	Transport	TCP, UDP — end to end delivery
3	Network	IP addressing, routing — IP, ICMP
2	Data Link	MAC, frames, ARP / Ethernet
1	Physical	Bits — cables, fiber, radio

L7-L5 → Data
 L4 → Segment / Datagram
 L3 → Packet
 L2 → Frame
 L1 → Bits

6. PDU — must memorize!

Encapsulation (sender): Data → Segment → Packet → Frame → Bits
 Receiver just reverses it = **decapsulation**.

7. TCP/IP Model

TCP/IP	≈ OSI
Application	L5 - L7
Transport	L4
Internet	L3
Network Access	L1 - L2

OSI = reference model. TCP/IP = the model the real Internet runs on.

8-10. IP Addressing

IPv4 = 32 bits, 4 octets, each 0-255. e.g. 192.168.1.10

Private IP Ranges ★★★★★ (memorize!)

10.0.0.0/8
172.16.0.0/12
192.168.0.0/16

Private → used inside private networks **Public** → used on the internet

Laptop 192.168.1.10 → Router/NAT → Public IP → Internet

11-12. Subnetting & CIDR ★★★★★

Subnetting = splitting one big network into smaller ones. Gives: better IP use, segmentation, smaller broadcast domains, easier management, more control.

192.168.1.0/24 → /24 = 24 network bits, 8 host bits → mask 255.255.255.0

13. Host Calculation

Total = $2^{\text{(host bits)}}$ | Usable = $2^{\text{(host bits)}} - 2$ (minus network + broadcast)

CIDR	Total	Usable
/24	256	254
/25	128	126
/26	64	62
/27	32	30
/28	16	14
/29	8	6
/30	4	2

14. Subnetting Example

Given 192.168.10.0/24, need 4 subnets → $2^2 = 4$ → borrow 2 bits → /24 becomes /26

192.168.10.0/26
192.168.10.64/26
192.168.10.128/26
192.168.10.192/26

First subnet → Network: 192.168.10.0 | Usable: .1 - .62 | Broadcast: .63

15-17. Ports & Sockets ★★★★★

IP identifies the machine; port identifies the service. e.g. 192.168.1.10:443

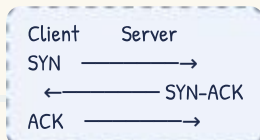
Port	Service
20/21	FTP
22	SSH
23	Telnet
25	SMTP
53	DNS
80	HTTP
110 / 143	POP3 / IMAP
443	HTTPS
3306 / 5432	MySQL / PostgreSQL

Socket = Protocol + IP + Port, e.g. TCP 192.168.1.10:443. Check with `ss -tlnp`

18-19. TCP vs UDP ★★★★★

TCP

- Connection-oriented, reliable
- Ordered, uses ACKs
- Retransmits lost data
- Flow / congestion control



UDP

- Connectionless, low overhead
- No guaranteed delivery/order
- Used by: DNS, video calls, VoIP, gaming, QUIC/HTTP3

Not just "bad TCP" — low overhead is a feature!

20. DNS ★★★★★

google.com → DNS → IP address

Record	Meaning
A / AAAA	IPv4 / IPv6
CNAME	Alias
MX	Mail server
NS	Name server
TXT	Text info

21-24. HTTP, SSH & Friends

HTTPS = HTTP over TLS — HTTP is still L7, TLS just adds security.

Status: 2xx success · 3xx redirect · 4xx client err · 5xx server err — know 200/301/400/401/403/404/500/502/503

SSH → port 22 → `ssh user@server-ip` · debug → `ssh -vvv` · SCP = secure copy over SSH

Real-World Troubleshooting

Don't randomly run commands — think in a sequence:

SYMPTOM → WHERE'S THE FAILURE? → COLLECT EVIDENCE
→ IDENTIFY ROOT CAUSE → FIX → VERIFY

25. Troubleshoot by OSI Layer

Layer	Might fail	Tools
L1	Cable / NIC / link	ip link, ethtool
L2	ARP / MAC / VLAN	ip neigh, arp
L3	IP / routing	ip a, ip route, ping, mtr
L4	Ports / TCP / UDP	ss, nc, nmap
L5-6	TLS / session	openssl
L7	DNS / HTTP / app	dig, curl, logs

Scenario snapshots

- **SSH fails** → ping → nc -zv :22 → ss -tlnp → systemctl status sshd → ufw status → ssh -vvv
- **Ping ok, "google.com" fails** → it's DNS → check /etc/resolv.conf, dig, nslookup, try dig @8.8.8.8
- **Site won't load** → ping → dig → curl -v → ss -tlnp :80/443 → systemctl status nginx → journalctl
- **Ping works, port doesn't** → host reachable at network layer, but TCP service isn't → check service/firewall/wrong port
- **localhost works, server IP doesn't** → check if app bound to 127.0.0.1 (local only) vs 0.0.0.0 (all interfaces)
- **Connection refused** → usually nothing is listening → ss -tlnp, systemctl status, journalctl (don't just blame the firewall — a firewall drop is usually a timeout, not a refusal)
- **502 Bad Gateway** → proxy can't reach backend → check nginx, curl the backend port, ss -tlnp, app logs
- **503 Service Unavailable** → app/backend/load-balancer overloaded or unhealthy → check health & logs

27. Cheat Sheet

Problem	Check first
No IP / no route	ip addr, ip route
Can't reach host	ping, mtr
DNS failure	dig, nslookup, resolv.conf
Port inaccessible	nc, ss
Service down	systemctl status
HTTP / TLS problem	curl -v, openssl s_client
Packet problem	tcpdump

28. The DevOps Troubleshooting Mindset

Don't say "I'll just restart the server." Instead:

```

What's the symptom?
↓
Which layer could cause it?
↓
What evidence can I collect?
↓
What does that evidence eliminate?
↓
What is the root cause?
↓
Fix it → Verify
  
```

Capture evidence **before** changing things. Check logs & monitoring. Document network info. Keep runbooks.

🔗 15 Questions You Should Answer (no notes!)

- What is a network?
- Hub vs switch vs router?
- MAC vs IP?
- The 7 OSI layers?
- What is encapsulation?
- OSI vs TCP/IP?
- What is an IP address?
- Public vs private IP?
- What does /24 mean?
- How do you calculate hosts in a subnet?
- TCP vs UDP?
- What is DNS?
- What is a port?
- What is a socket?
- How would you troubleshoot a server reachable by ping but not by SSH/HTTP?

If you can answer #15 properly — you're starting to think like a DevOps engineer, not just memorizing definitions. 🧠